

CO5_ASSESSMENT 1

CONSTRAINT-BASED PROBLEM SOLVING

1. Real-Time Face Recognition for Campus Security

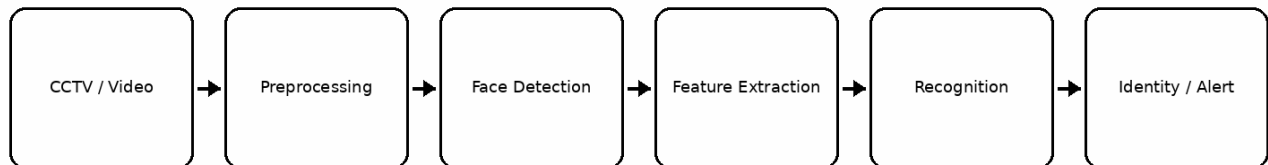
A university wants to deploy a camera-based security system at the entrance that can identify registered students and staff in real time.

The system should:

- Capture faces from live CCTV/video streams.
- Detect faces under different lighting conditions.
- Extract suitable facial features.
- Recognize registered individuals.
- Maintain real-time performance on a low-cost computer.
- Minimize false recognition.

Design a complete OpenCV-based face recognition pipeline from image acquisition to final identification. Explain the preprocessing, face detection, feature extraction, recognition, and optimization techniques used. Justify how your system achieves accuracy and real-time performance.

REAL-TIME FACE RECOGNITION



Illustrative OpenCV pipeline

Answer:

A complete OpenCV-based campus face recognition system can be designed as:

Image Acquisition → Preprocessing → Face Detection → Feature Extraction → Face Recognition → Decision → Display/Alert.

1. Image Acquisition:

The CCTV camera continuously captures live video using OpenCV VideoCapture(). The frame resolution can be reduced to improve processing speed.

2. Preprocessing:

Convert the frame to grayscale, apply Gaussian or median filtering to remove noise, and use histogram equalization or CLAHE to improve visibility under poor lighting. Resize frames when required.

3. Face Detection:

Use Haar Cascade for a lightweight solution or an OpenCV DNN face detector for better robustness. The detector identifies the location of every face and creates bounding boxes.

4. Feature Extraction:

Crop each detected face, resize it to a fixed size, normalize illumination, and extract discriminative facial features. LBPH can be used for a lightweight OpenCV implementation. For higher accuracy, deep facial embeddings can be generated using an OpenCV DNN model.

5. Face Recognition:

Compare the extracted features with registered student and staff features. If the distance is below a selected threshold, the individual is recognized; otherwise, the person is classified as Unknown.

6. Minimizing False Recognition:

Use multiple registration images, different poses and lighting conditions, a strict recognition threshold, face-quality checks, and confirmation across several consecutive frames.

7. Real-Time Optimization:

Resize input frames, detect faces at lower resolution, process every second or third frame, track already detected faces between detection frames, and use lightweight models. These techniques reduce CPU usage and processing delay.

8. Output:

Display a bounding box, person's name, recognition confidence, and an Unknown Person warning when appropriate.

Thus, the combination of preprocessing, reliable detection, feature matching, temporal verification, and frame-level optimization provides accurate recognition while maintaining real-time performance on a low-cost computer.

2. Intelligent Traffic Violation Detection System

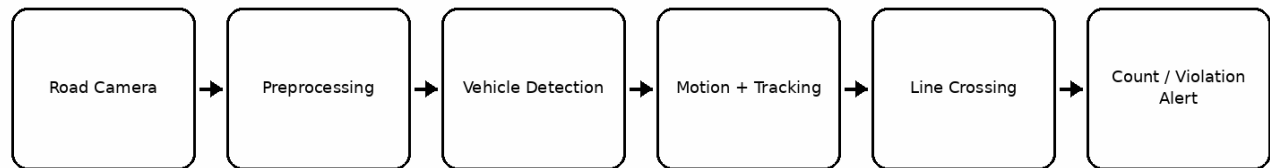
A traffic department wants to automatically monitor vehicles using roadside cameras.

The system should:

- Detect vehicles from live video.
- Track vehicles across consecutive frames.
- Identify vehicles crossing a predefined line.
- Count vehicles accurately.
- Detect abnormal or unwanted movements.
- Display vehicle count and alerts in real time.

Design an OpenCV-based solution incorporating video acquisition, preprocessing, object detection, motion analysis, tracking, and visualization. Explain how you would handle shadows, camera noise, dynamic backgrounds, and multiple vehicles to minimize counting errors.

INTELLIGENT TRAFFIC MONITORING



Illustrative OpenCV pipeline

Answer:

The traffic violation system can use the following OpenCV pipeline:

Video Acquisition → Preprocessing → Vehicle Detection → Motion Analysis → Tracking → Line Crossing → Violation Detection → Counting → Visualization.

1. Video Acquisition:

Capture continuous roadside CCTV video using OpenCV VideoCapture(). A fixed camera position provides a stable background.

2. Preprocessing:

Resize frames, apply Gaussian or median filtering, and adjust contrast when necessary. This reduces camera noise and improves detection.

3. Vehicle Detection:

For a fixed camera, background subtraction such as MOG2 or KNN can detect moving vehicles. For better classification, an OpenCV DNN object detector can identify cars, buses, trucks, and motorcycles.

4. Motion Analysis:

Compare the current frame with the estimated background to produce a foreground mask. Morphological opening removes small noise, while closing and dilation connect fragmented vehicle regions.

5. Vehicle Tracking:

Assign a unique tracking ID to every detected vehicle. Tracking maintains the identity of a vehicle across consecutive frames and prevents repeated counting.

6. Line Crossing:

Draw a virtual line across the road. Calculate the center point of each tracked vehicle. When the center moves from one side of the line to the other, increase the vehicle count once.

7. Abnormal Movement:

Detect wrong-direction movement, sudden stopping, restricted-lane entry, or unusual trajectories using the tracked vehicle's position and movement direction.

8. Handling Shadows and Dynamic Backgrounds:

Use shadow detection, contour-area filtering, morphological processing, and adaptive background models. Small moving objects such as leaves and minor shadows can be rejected using minimum-area and persistence conditions.

9. Multiple Vehicles:

Each vehicle receives a separate tracking ID. Position, centroid, and movement history are used to maintain identity and prevent duplicate counts.

10. Visualization:

Display bounding boxes, vehicle IDs, virtual line, vehicle count, and violation alerts in real time.

Therefore, the system combines object detection, motion analysis, tracking, and line-crossing logic to provide reliable traffic counting and violation detection while reducing errors caused by shadows, noise, and multiple vehicles.

3. Automated Document Processing System

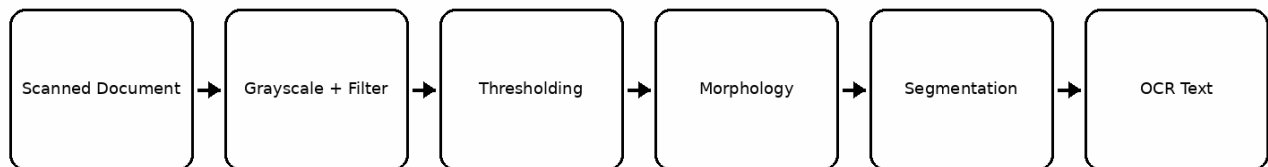
A company receives thousands of scanned documents that may contain blurred text, uneven illumination, noise, and different font sizes.

The system should:

- Automatically improve document quality.
- Remove noise and unwanted background.
- Convert the document into a suitable format for OCR.
- Segment text regions.
- Extract readable text efficiently.
- Process a large number of documents in batch mode.

Design an OpenCV-based image processing pipeline for this system. Explain the role of grayscale conversion, thresholding, filtering, morphological operations, segmentation, and geometric correction. Justify how each stage improves OCR accuracy.

AUTOMATED DOCUMENT PROCESSING



Illustrative OpenCV pipeline

Answer:

The automated document processing pipeline is:

Document Input → Grayscale Conversion → Noise Removal → Illumination Correction → Thresholding → Morphological Processing → Text Segmentation → Geometric Correction → OCR.

1. Image Acquisition:

Documents may come from scanners, mobile cameras, or digital images. Each page is converted into an image for processing.

2. Grayscale Conversion:

Convert the color document to grayscale. This reduces the three color channels to one intensity channel and simplifies subsequent processing.

3. Noise Removal:

Use median filtering to remove salt-and-pepper noise or Gaussian filtering for general noise reduction. This improves character clarity without significantly damaging text edges.

4. Illumination Correction:

Use CLAHE, background estimation, or adaptive methods to reduce the effect of uneven lighting and improve local contrast.

5. Thresholding:

Convert the grayscale image into a binary image using Otsu or adaptive thresholding. Adaptive thresholding is especially useful when different parts of the page have different brightness levels.

6. Morphological Operations:

Opening removes small unwanted objects. Closing fills gaps in characters. Dilation strengthens weak text, while erosion can remove small unwanted connections.

7. Text Segmentation:

Use contours or connected-component analysis to locate text blocks, lines, words, or characters. This allows OCR to focus on meaningful text regions.

8. Geometric Correction:

Detect document boundaries and correct skew, rotation, and perspective distortion using rotation and perspective transformation.

9. OCR Preparation:

The final image should contain clear, aligned, high-contrast characters with minimal background noise. It is then supplied to an OCR engine.

10. Batch Processing:

Thousands of documents can be processed automatically by reading each document, applying the same preprocessing pipeline, performing OCR, saving the extracted text, and moving to the next document.

The main benefit is that every stage improves the quality of the image before OCR. This increases character readability, reduces recognition errors, and supports efficient large-scale document processing.

4. Real-Time Multi-Person Tracking and Attendance System

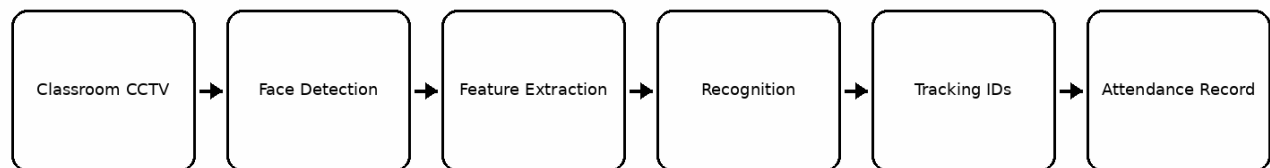
A college wants to automatically record student attendance using classroom CCTV cameras.

The system should:

- Detect multiple faces simultaneously.
- Track each person across video frames.
- Recognize registered students.
- Avoid repeatedly marking the same student.
- Handle slight head movement and temporary face disappearance.
- Operate continuously with minimum processing delay.

Design a complete OpenCV-based system using face detection, feature extraction, recognition, and tracking. Explain how you maintain identity across frames and prevent false or duplicate attendance records.

MULTI-PERSON TRACKING & ATTENDANCE



Illustrative OpenCV pipeline

Answer:

The multi-person attendance system can be designed using:

CCTV Video → Preprocessing → Face Detection → Feature Extraction → Face Recognition → Multi-Person Tracking → Identity Management → Attendance Validation → Database.

1. Video Acquisition:

Capture classroom CCTV video continuously using OpenCV. Reduce frame resolution if necessary to improve real-time performance.

2. Preprocessing:

Resize the frame, convert to grayscale when appropriate, remove noise, and improve contrast using techniques such as CLAHE.

3. Face Detection:

Detect all visible faces in each frame using Haar Cascade or an OpenCV DNN face detector. Each detected face receives a bounding box.

4. Feature Extraction:

Crop each face, resize it, normalize it, and extract facial features. LBPH is suitable for a lightweight OpenCV implementation, while deep embeddings can provide stronger recognition.

5. Face Recognition:

Compare the extracted features with registered student data. If the similarity is sufficient, assign the corresponding student ID. Otherwise, mark the person as Unknown.

6. Multi-Person Tracking:

Assign a unique tracking ID to each person. Centroid tracking or another suitable tracker maintains each person's position between frames.

7. Maintaining Identity:

Use previous location, bounding-box movement, facial features, and appearance information to associate the current detection with an existing track. A short-term tracking period can handle temporary face disappearance.

8. Preventing Duplicate Attendance:

Maintain a set or database record of students who are already marked present. If the same student is recognized again, the system does not create another attendance entry.

9. Reducing False Attendance:

Do not mark attendance from one uncertain frame. Require recognition with sufficient confidence across multiple consecutive frames before recording attendance.

10. Real-Time Optimization:

Use frame skipping, smaller input frames, lightweight recognition models, and tracking between face-detection frames. Process only face regions rather than the complete image whenever possible.

11. Output:

Display face bounding boxes, student names, tracking IDs, confidence values, present count, and attendance status.

Thus, tracking maintains identity across frames, while temporal confirmation and duplicate checking ensure that each registered student is marked only once and false attendance is minimized.

5. Smart Home Surveillance and Intrusion Detection

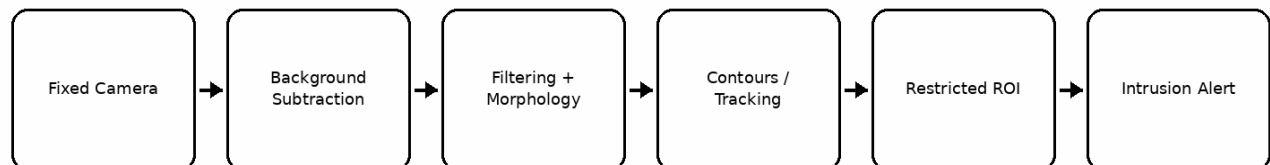
A home security system uses a fixed camera to monitor an entrance continuously.

The system should:

- Detect significant motion.
- Distinguish actual movement from camera noise and small background changes.
- Track moving objects.
- Identify suspicious activity.
- Generate an alert when an unauthorized person enters a predefined region.
- Operate in real time with limited computational resources.

Design an OpenCV-based surveillance pipeline using background subtraction, filtering, morphological operations, contour/feature detection, motion analysis, and tracking. Explain how your approach reduces false alarms caused by shadows, illumination changes, trees, and other dynamic background objects.

SMART HOME SURVEILLANCE



Illustrative OpenCV pipeline

Answer:

The smart home surveillance pipeline is:

Camera → Frame Acquisition → Background Subtraction → Noise Filtering → Morphological Processing → Contour Detection → Object Tracking → ROI Analysis → Intrusion Decision → Alert.

1. Video Acquisition:

A fixed camera continuously monitors the entrance. OpenCV VideoCapture() reads the live frames.

2. Background Subtraction:

Since the camera is fixed, MOG2 or KNN background subtraction can estimate the static background and identify foreground motion.

3. Filtering:

Apply Gaussian or median filtering to reduce sensor noise. Very small foreground regions are rejected using a minimum-area threshold.

4. Morphological Operations:

Opening removes isolated noise, closing fills gaps, and dilation strengthens connected moving regions. This creates cleaner foreground objects.

5. Contour Detection:

Extract contours from the processed foreground mask. For each contour, calculate its bounding rectangle, area, and centroid. Very small contours are ignored.

6. Motion Analysis:

Compare object positions across consecutive frames. Persistent and sufficiently large movement is treated as meaningful motion rather than random camera noise.

7. Object Tracking:

Assign a unique ID to each moving object and update its position over time. Tracking prevents the same object from being treated as a new object in every frame.

8. Restricted Region:

Define a Region of Interest (ROI) around the entrance. If a tracked object enters this region, the system checks whether it represents a possible intrusion.

9. Intrusion Decision:

A possible alert can be generated when motion is detected, the object has sufficient size, the object persists for several frames, and its position enters the restricted ROI.

10. Reducing False Alarms:

Shadows can be reduced using shadow detection, area filtering, and shape analysis. Illumination changes can be handled using adaptive background models. Trees, leaves, curtains, and other dynamic objects can be reduced using ROI restriction, minimum contour area, persistence checks, and tracking.

11. Real-Time Optimization:

Reduce frame resolution, process selected frames, use lightweight background subtraction, restrict processing to the entrance ROI, and track objects instead of repeatedly detecting them from scratch.

12. Visualization:

Display the live video, foreground regions, object bounding boxes, tracking IDs, restricted ROI, intrusion status, and alert messages.

Therefore, the system can distinguish significant movement from noise and dynamic background changes while operating continuously with limited computational resources.

